

AWS EDITION

AWS Agentic AI Compute Node Runbook

Secure banking-agent proof of concept on Amazon Bedrock + Aurora PostgreSQL / pgvector

Amazon Bedrock	Aurora PostgreSQL + pgvector	IAM / SSM / Secrets
Generation + embeddings	Facts + vector retrieval + audit	Workload identity + secure access

Design principle: AI can investigate, assemble evidence, interpret authorized policy, and recommend the next action - while the human remains accountable for the consequential decision.

1. Scope and target architecture

This runbook builds a portfolio-safe AWS runtime for the synthetic banking agent. The recommended demonstration pattern is an Amazon EC2 application node accessed through AWS Systems Manager Session Manager, an Aurora PostgreSQL-compatible or RDS PostgreSQL database with pgvector, and Amazon Bedrock for embeddings and generation.

Production boundary: this is a proof of concept. Do not place real banking data, production credentials, or regulated workloads into the demo environment.

Component	Demo implementation	Why
Application node	Amazon EC2, Amazon Linux 2023 or equivalent	Simple interactive host for Python demo and troubleshooting
Admin access	Systems Manager Session Manager	Avoids exposing inbound SSH for routine administration
Database	Aurora PostgreSQL-compatible / RDS PostgreSQL	Relational facts, action state, audit, and pgvector
AI	Amazon Bedrock	Managed model inference without running a local LLM
Secrets	AWS Secrets Manager	Keeps database credentials out of source control

2. IAM and access prerequisites

Create or reuse an EC2 instance role. The application needs permission to invoke the selected Bedrock models and, when configured, read the specific database secret. For Session Manager, the instance must also be configured as a managed node.

Minimum application permissions

```
{
  "Effect": "Allow",
  "Action": ["bedrock:InvokeModel"],
  "Resource": "*"
}

{
  "Effect": "Allow",
  "Action": ["secretsmanager:GetSecretValue"],
  "Resource": "arn:aws:secretsmanager:*:*:secret:bankdemo-*"
}
```

For a production environment, scope model and secret resources more tightly and separate deployment permissions from runtime permissions.

3. Launch and connect to the EC2 node

- Launch an EC2 instance in the same VPC as the database or with appropriate routed connectivity.
- Attach the instance role before running the application.
- Prefer Session Manager for administrative access.
- Allow outbound HTTPS (443) for AWS API calls, or configure appropriate VPC endpoints.
- Do not add a broad 0.0.0.0/0 SSH rule solely for the demo.

Session Manager workflow

```
AWS Console -> Systems Manager -> Session Manager -> Start session
```

```
# Once connected
whoami
python3 --version
aws sts get-caller-identity
```

4. Prepare the application environment

```
sudo dnf update -y
sudo dnf install -y git python3 python3-pip

git clone <your-repository-url>
cd secure-agentic-ai-banking-demo
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
cp .env.example .env
chmod 600 .env
```

Do not place AWS access keys in `.env`. On EC2, boto3 uses the instance role automatically through the standard AWS credential provider chain.

5. Configure Aurora PostgreSQL / RDS PostgreSQL

- Keep the database non-public for the recommended demo topology.
- Permit TCP 5432 inbound only from the EC2 application security group.
- Create the application database and a least-privileged demo owner/user.
- Enable the `vector` extension and apply the repository schema and fictional data.

```
CREATE EXTENSION IF NOT EXISTS vector;
```

```
-- Then apply:
sql/schema.sql
sql/synthetic_data.sql
sql/policy_chunks.sql
```

The AWS edition stores policy embeddings in `vector(1024)` because the default project configuration uses Titan Text Embeddings V2 with 1024 dimensions.

6. Configure secrets and environment variables

Preferred: store the PostgreSQL host, username, password, port, and database name in Secrets Manager and set only `DB_SECRET_ARN` in `.env`. Local development can use `DB_*` variables, but the file must remain uncommitted.

```
AWS_REGION=us-east-1
BEDROCK_GEN_MODEL=amazon.nova-lite-v1:0
BEDROCK_EMBED_MODEL=amazon.titan-embed-text-v2:0
EMBED_DIMENSIONS=1024
DB_SECRET_ARN=arn:aws:secretsmanager:REGION:ACCOUNT:secret:bankdemo-...
EMPLOYEE_ID=CSR001
EMPLOYEE_ROLE=CUSTOMER_SERVICE
```

7. Validate Bedrock and database connectivity

```
source .venv/bin/activate
python src/test_bedrock.py
python src/test_db.py
```

- The Bedrock test should return an embedding with the configured dimensions and a short generation response.
- The database test should display PostgreSQL information and confirm that pgvector is installed.

8. Load vectors and run the demo

```
python src/embed_policies.py
python src/search_policy.py
./scripts/reset_demo.sh
python src/bank_agent.py 847291
python src/review_action.py
```

Expected security result: CUSTOMER_SERVICE can retrieve CV-04 and CA-12 but not restricted FR-09. The agent creates a PENDING action and stops until a human approves or rejects it.

9. Troubleshooting checklist

Symptom	Check
AccessDenied from Bedrock	Instance role permissions, model/Region availability, correct model ID
Embedding dimension mismatch	EMBED_DIMENSIONS and PostgreSQL vector column size must match
Database timeout	VPC routing, security groups, DNS, database status
Secret access denied	Secret ARN and IAM scope
Session Manager unavailable	Instance profile, SSM Agent status, outbound connectivity / endpoints
Restricted policy appears	Role predicate in search_policy.py and allowed_role data

10. Production hardening

- Use private service endpoints where appropriate and formal network segmentation.
- Apply KMS encryption and rotation policies for secrets and data.
- Move the application runtime to a managed container platform when scale or deployment automation requires it.
- Add CloudWatch logs/metrics/traces, alarms, and SIEM integration.
- Implement formal model and RAG evaluation, prompt-injection tests, and policy compliance tests.
- Move consequential execution behind authenticated, idempotent action APIs and segregation-of-duties controls.

AWS reference sources

- AWS: Nova Lite model card - <https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-lite.html>
- AWS: Titan Text Embeddings models - <https://docs.aws.amazon.com/bedrock/latest/userguide/titan-embedding-models.html>
- AWS: Aurora PostgreSQL vector database - <https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/AuroraPostgreSQL.VectorDB.html>
- AWS: Systems Manager Session Manager - <https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>